



WACHEMO UNIVERSITY ENGINEERING AND TECHNOLOGY DEPARTMENT OF INFORMATION TECHNOLOGY

COURSE TITLE FUNDAMENTALS OF DATABASE

Group members	ID
1,Bruki Tedesse.....	172717
2 Cherinet Tessema.....	170972
3,Derebe Zeleka	176064
4,Henok Brihanu.....	176534
5,Hlkima Mohammed.....	171637

Project Title

Wachemo University Students Clinic Management System

Introduction

This database system is designed to digitize the daily operations of the Wachemo University Student Clinic.

The importance of a digital health record system for university clinics.

- The purpose: To store, manage, and retrieve student/staff medical information efficiently.**
- Benefits: Reduces paperwork, speeds up service, ensures accurate prescriptions, tracks medicine stock, etc.**
- Mention Wachemo University's student population and need for organized health services.**

The system manages student registration, medical history, laboratory tests, and pharmacy dispensing.

Its primary goal is to reduce paperwork and provide fast, accurate medical services to students.

Problem Description

Describe what the current system lacks (e.g., manual records, slow retrieval, no central database).

****Users:****

- **Students/Staff (Patients)**
- **Doctors/Nurses**
- **Clinic Administrators**

****Data stored:****

Patient details, visits, diagnoses, prescriptions, inventory, appointments.

Problems solved:

- 1. Manual record-keeping errors.**
- 2. Difficulty in tracking patient history.**
- 3. Poor medicine inventory management.**
- 4. Long appointment wait times.**

*** What the system is about: The system is a data management tool for the university clinic's workflow.**

*** Who uses it (Users): Receptionists (Card Office), Doctors, Laboratory Technicians, and Pharmacists.**

*** What data is stored: Student identities, medical records (diagnosis), laboratory results, and medicine inventory.**

*** What problems the database solves: It prevents the loss of paper files, allows for quick retrieval of patient history, and helps monitor pharmacy stock levels.**

Objectives

List measurable goals:

- 1. Design a secure, normalized relational database for the clinic.**
- 2. Enable easy appointment scheduling.**
- 3. Maintain patient medical history.**
- 4. Manage medicine stock and expiry dates.**
- 5. Generate reports (daily patients, stock levels, doctor schedules).**

- * To organize student information in a centralized database.**
- * To simplify communication between doctors and patients.**
- * To link laboratory and pharmacy workflows directly to the patient's file.**
- * To generate medical reports easily.**

Scope of the Project

Included

The project includes student registration, diagnosis, laboratory services, and pharmacy management.

- Patient registration (students/staff)**
- Appointment management**
- Medical visit recording**
- Prescription & inventory management**
- Basic user roles (Doctor, Nurse, Admin)**

Exclude

It excludes external referral clinical processes and staff payroll management.

Laboratory management

- Multi-branch clinic support**

Requirements Analysis

Based on the guideline requirements for entities, attributes, and relationships:

Entities & Attributes:

*** Student: StudentID, FullName, Gender, Department, Dorm_Number, Phone.**

- * **Doctor:** DoctorID, Name, Specialization, Working_Shift.
- * **Medical_Record:** Record ID , Date, Symptoms, Diagnosis.
- * **Lab_Test:** LabID, Test_Type, Result, Lab_Technician.
- * **Medicine:** DrugID, Drug_Name, Expiry_Date, Quantity.

Relationships:

- * **Student -to- Record:** One-to-Many (A student can have multiple medical visits).
- * **Doctor -to- Record:** One-to-Many (A doctor can treat many students).
- * **Record -to- Lab:** One-to-One/Many (A medical record can trigger lab tests).
- * **Record -to- Medicine:** Many-to-Many (One record can have many medicines; one medicine can be in many records).

ER Diagram (Logic)

The ER Diagram will visualize these Entities (Rectangles), Attributes (Ovals), and Relationships (Diamonds) with their respective cardinalities (1:M)

erDiagram

STUDENT {

```
    int StudentID PK
    string FullName
    string Gender
    string Department
    string Dorm_Number
    string Phone
}
```

```
DOCTOR {
    int DoctorID PK
    string Name
    string Specialization
    string Working_Shift
}
```

```
MEDICAL_RECORD {
    int RecordID PK
    date Date
    string Symptoms
    string Diagnosis
}
```

}

LAB_TEST {

int LabID PK

string Test_Type

string Result

string Lab_Technician

}

MEDICINE {

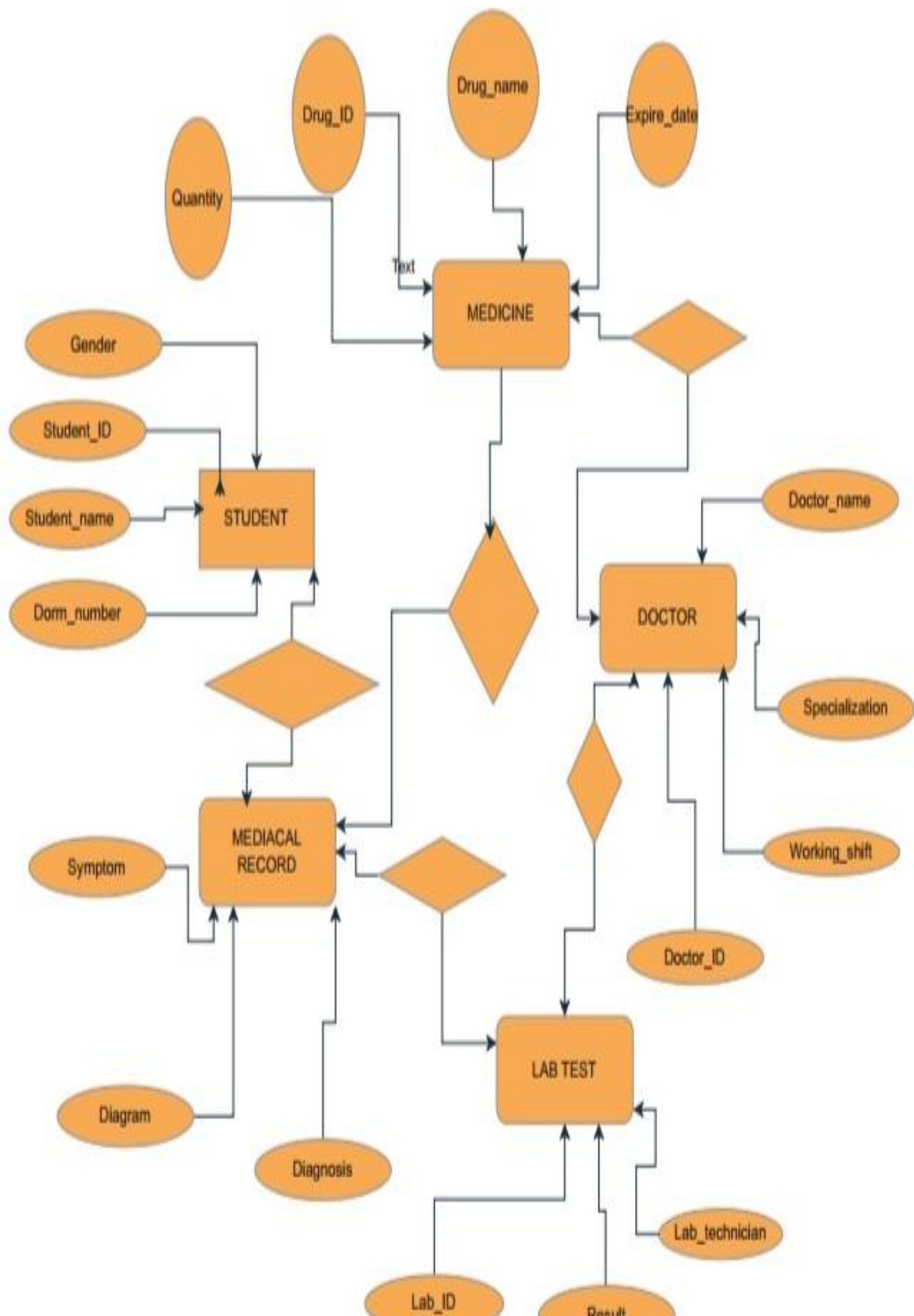
int DrugID PK

string Drug_Name

date Expiry_Date

int Quantity

}



SQL Implementation

-- Database creation

```
CREATE DATABASE WCU_Clinic_DB;
```

```
USE WCU_Clinic_DB;
```

-- Students Table

```
CREATE TABLE Students (
```

```
    StudentID VARCHAR(20) PRIMARY KEY,
```

```
    FullName VARCHAR(100) NOT NULL,
```

```
    Gender CHAR(1),
```

```
    Dept VARCHAR(50),
```

```
    DormNo VARCHAR(20)
```

```
);
```

-- Doctors Table

```
CREATE TABLE Doctors (
```

```
    DoctorID INT PRIMARY KEY AUTO_INCREMENT,
```

```
    DoctorName VARCHAR(100),  
    Specialization VARCHAR(50)  
);  
  
-- Medical Records Table  
  
CREATE TABLE MedicalRecords (  
    RecordID INT PRIMARY KEY AUTO_INCREMENT,  
    StudentID VARCHAR(20),  
    DoctorID INT,  
    Diagnosis TEXT,  
    TreatmentDate DATE,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (DoctorID) REFERENCES Doctors(Doctor
```

Normalization (Up to 3rd Normal Form)

Normalization is essential to reduce data redundancy and improve data integrity.

*** 1st Normal Form (1NF): We ensure that all table cells contain atomic (single) values and there are no repeating groups. For example, a student cannot have multiple phone numbers in one cell; they must be separated or stored in a way that avoids multi-valued attributes.**

*** 2nd Normal Form (2NF): We ensure the table is in 1NF and all non-key attributes are fully functional and dependent on the Primary Key. For instance, in the Medical_Records table, the Diagnosis depends entirely on the RecordID**

.Student table

****1NF (First Normal Form)****

Eliminate repeating groups - medications

****MEDICAL_RECORD Table (1NF):****

Student-name	Student-ID	Department	Doctor-name	Doctor ID	Medication name	Medication cost /ID	Visit date	Diagnosis
Matavisa Masha	144300	Civil	Bethlehem tedela	2265	Amoxicillin	500	23/01/2026	Common
Latebo Kebede	184741	Freshman	Befekadu Matheowos	3683	Albuterol	450	23/01/2026	Common cold
Etenash Hailu	171014	Comprehensive	Laka Lalago	3630	Kisinopril	40	23/01/2026	Common cold
Harreya Shaminsu	151856	Public health	Abreham Bafakadu	3620	Nichol	50	23/01/2026	Join pain
Emabet Teka	185240	Freshman	Tedela Jabamo	2540	Fluoxetine	20	23/01/2026	Mental

**2NF (Second Normal Form)**

Remove partial dependencies - separate entities

****STUDENT Table:**

Student ID	Student Fname	Student Lname	Department
1443000	Matayisa	MAsgard	Civil
184741	Latebo	Kebede	Freshman
171014	Etenash	Hailu	Comprehensive
151816	Hareyya	Shaminsu	Public health
185240	Emabet	Taka	Freshman



Doctor table name

Doctor ID	Doctor fname	Doctor lname
2265	Bethlehem	Tedela
3683	Befekadu	Matheowos
3630	Leka	Lalago
3620	Abreham	Bafakadu
2540	Tedela	Jabamo

Medication table

Medication name	Medication cost
Amoxicillin	500
Albuterol	450
Lisinopril	40
Dichlo	50
Fluoxetine	20

Visit table

Visit ID	Student ID	Doctor ID	Visit date	Diagnosis	Status
V001	144300	2265	23/01/2026	Common cold	Completed

Prescription table

Prescription ID	Visit ID	Medicaion name	Quantity	Duration	Total cost
-----------------	----------	----------------	----------	----------	------------

*** 3rd Normal Form (3NF):** We ensure the table is in 2NF and remove transitive dependencies. This means a non-key attribute should not depend on another non-key attribute. For example, we separate DepartmentName into a different table so it doesn't depend on a StudentID.

Student :

Student name	Student ID	Department
--------------	------------	------------

Doctor table the same 2NF

Department :

Department code	Department name
Civil	Civil engineering
Fresh	Freshman
Comp	Comprehensive

Visit table

Visit ID	Student ID	Dr ID	Visit date	Diagnosis
----------	------------	-------	------------	-----------

Prescription table

Complete 3NF Scheme :

Student table

Prescription id	Visit ID	Total cost	Mediation name	Dosage
P001	V001	500	<u>Amoxicilin</u>	500mg
P002	V002	450	Albuterol	2
P003	V003	40	<u>Lisinoperil</u>	40
P004	V004	50	<u>Dichlo</u>	50

Student name	Departement ID	Department
Matayida	Civil	Civil
Latebo	Freshman	Freshman
Comprehensive	Comprehensive	Comprehensive

Department table

Department code	Department name
Civil	Civil engineering
Fresh	Freshman
Comp	Comprehensive

Doctor table

Doctor name	Doctor ID
Bethlehem Tedela	2265
Bafikadu	3683

Medication name	Medication ID	Category	<u>Manufacturor</u>
<u>Amoxicilin</u>	500mg	Antibiotics	Pharmacy
Albuterol	45mg	Bronco	Pharmacy

Medication tabl

Visit table

Visit ID	Student ID	Doctor ID	Visit date	Diagnosed6	Status
V001	144300	2265	23/01/2026	Common	Completed

				c!ld	
--	--	--	--	------	--

Prescription table

Prescription ID	Visit ID	Medicaion name	Quantity	Duration	Total cost
-----------------	----------	----------------	----------	----------	------------

****Summary of Normalization:****

1. ****UNF → 1NF:**** Separated repeating medication groups into individual rows

2. ****1NF → 2NF:****

- Separated Student, Doctor, Medication entities
- Created Visit table for student-doctor encounters
- Created Prescription table for visit-medication relationships

3. ****2NF → 3NF:****

- Created Department table (optional)
- Ensured no transitive dependencies exist
- Added proper foreign key relationships

****Foreign Key Relationships:****

- STUDENT.Dept_Code → DEPARTMENT.Dept_Code
- VISIT.Student_id → STUDENT.Student_id
- VISIT.Dr_ID → DOCTOR.Dr_ID
- PRESCRIPTION.Visit_ID → VISIT.Visit_ID
- PRESCRIPTION.Medication_name →
MEDICATION.Medication_name

Database Implementation (SQL)

Here is the full SQL code to create your database with constraints, sample data, and the required queries.

-- 1. STUDENT Table

CREATE TABLE STUDENT (

StudentID INT PRIMARY KEY,

```
FullName VARCHAR(100) NOT NULL,  
Gender CHAR(1) NOT NULL CHECK (Gender IN ('M', 'F', 'O')),  
DepartmentID INT NOT NULL,  
YearOfStudy INT NOT NULL CHECK (YearOfStudy BETWEEN 1  
AND 6),  
Phone VARCHAR(15) UNIQUE NOT NULL,  
CONSTRAINT fk_student_department FOREIGN KEY  
(DepartmentID)  
REFERENCES DEPARTMENT(DepartmentID) ON DELETE  
RESTRICT  
);
```

```
-- =====
```

```
-- 2. DOCTOR Table
```

```
CREATE TABLE DOCTOR (  
    DoctorID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Specialization VARCHAR(50) NOT NULL,  
    Phone VARCHAR(15) UNIQUE NOT NULL,
```

```
WorkingShift VARCHAR(20) NOT NULL CHECK (WorkingShift  
IN ('Morning', 'Afternoon', 'Night', 'Full-day')),  
);
```

-- 3. DEPARTMENT Table

```
CREATE TABLE DEPARTMENT (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(100) UNIQUE NOT NULL,  
    DepartmentCode VARCHAR(10) UNIQUE NOT NULL,  
    Phone VARCHAR(15)  
);
```

-- 4. VISIT Table (Appointment/Consultation)

```
CREATE TABLE VISIT (  
    VisitID INT PRIMARY KEY AUTO_INCREMENT,  
    StudentID INT NOT NULL,  
    DoctorID INT NOT NULL,  
    VisitDate DATE NOT NULL,  
    VisitTime TIME NOT NULL,  
    VisitType VARCHAR(20) NOT NULL DEFAULT 'Consultation'  
    Diagnosis ,
```

Temperature DECIMAL(4,1),

BloodPressure VARCHAR(10),

PulseRate INT,

Status VARCHAR(20) DEFAULT 'Completed'

**CHECK (Status IN ('Scheduled', 'In Progress', 'Completed',
'Cancelled'))),**

CREATE TABLE MEDICATION (

MedicationID INT PRIMARY KEY AUTO_INCREMENT,

MedicationName VARCHAR(100) NOT NULL,

GenericName VARCHAR(100),

Category VARCHAR(50) NOT NULL.,

DosageForm VARCHAR(30),

Strength VARCHAR(50),

Manufacturer VARCHAR(100),

**QuantityInStock INT DEFAULT 0 CHECK (QuantityInStock >=
0),**

UnitPrice DECIMAL(10,2) CHECK (UnitPrice >= 0),

ExpiryDate DATE NOT NULL,

ReorderLevel INT DEFAULT 10 CHECK (ReorderLevel >= 0),


```
StorageInstructions VARCHAR(200),  
  
CONSTRAINT chk_expiry_date CHECK (ExpiryDate >  
CURRENT_DATE)  
  
);
```

-- 6. PRESCRIPTION Table

```
CREATE TABLE PRESCRIPTION (  
  
PrescriptionID INT PRIMARY KEY AUTO_INCREMENT,  
  
VisitID INT NOT NULL,  
  
MedicationID INT NOT NULL,  
  
Dosage VARCHAR(50) NOT NULL,  
  
Frequency VARCHAR(50) NOT NULL,  
  
Duration VARCHAR(50) NOT NULL,  
  
Quantity prescribed INT NOT NULL CHECK (Quantity  
Prescribed > 0),  
  
Instructions TEXT,  
  
Prescribed Date DATE DEFAULT CURRENT_DATE,  
  
FOREIGN KEY (VisitID) REFERENCES VISIT(VisitID) ON DELETE  
CASCADE,
```

```
FOREIGN KEY (MedicationID) REFERENCES  
MEDICATION(MedicationID) ON DELETE RESTRICT,  
  
CONSTRAINT unique_prescription_item UNIQUE (VisitID,  
MedicationID)  
  
);
```

-- 7. LAB_TEST Table

```
CREATE TABLE LAB_TEST (  
  
    TestID INT PRIMARY KEY AUTO_INCREMENT,  
  
    VisitID INT NOT NULL,  
  
    TestName VARCHAR(100) NOT NULL,  
  
    SampleCollectionDate DATE,  
  
    Result TEXT,  
  
    ResultValue DECIMAL(10,2),  
  
    LabTechnician VARCHAR(100),  
  
    Status VARCHAR(20) DEFAULT 'Pending'  
  
    FOREIGN KEY (VisitID) REFERENCES VISIT(VisitID) ON DELETE  
CASCADE
```

);

-- Indexes for Performance

**CREATE INDEX idx_student_department ON
STUDENT(DepartmentID);**

CREATE INDEX idx_student_phone ON STUDENT(Phone);

**CREATE INDEX idx_doctor_specialization ON
DOCTOR(Specialization);**

CREATE INDEX idx_visit_date ON VISIT(VisitDate);

**CREATE INDEX idx_visit_student ON VISIT(StudentID,
VisitDate DESC);**

**CREATE INDEX idx_medication_expiry ON
MEDICATION(ExpiryDate);**

**CREATE INDEX idx_prescription_visit ON
PRESCRIPTION(VisitID);**

**CREATE INDEX idx_labtest_status ON LAB_TEST(Status,
RequestedDate);**

-- Sample INSERT Statements for Testing

-- Insert Departments

**INSERT INTO DEPARTMENT (DepartmentID, DepartmentName,
DepartmentCode, Faculty) VALUES**

(101, 'Civil', 'Freshman ', 'Comprehensive ',

-- Insert Doctors

**INSERT INTO DOCTOR (DoctorID, Name, Specialization,
LicenseNumber, Phone, WorkingShift) VALUES**

**(201, 'Dr. Bethlehem ', 'General Physician', 'MD-ET-001',
'Morning'),**

(202, 'Befekadu', 'Pediatrics', 'MD-ET-002', 'Afternoon'),

(203, 'Dr. Tedele, 'Psychiatry', 'MD-ET-003', 'Full-day');

-- Insert Students

**INSERT INTO STUDENT (StudentID, FullName, Gender,
DepartmentID, YearOfStudy, DateOfBirth) VALUES**

(144300, 'Matayisa', 'M', 101, 3, '2002-05-15'),

(184741 'Latebo', 'F', 102, 2, "2003-08-22"),

(185240,'Emanet ', 'F',1003,3,'2001/12/03),

(40, 'Etenash hailu, 'M', 103, 4,, '2001-11-30');

-- Insert Medications

**INSERT INTO MEDICATION (MedicationName, Category,
DosageForm, Strength, QuantityInStock, UnitPrice, ExpiryDate)
VALUES**

**('Paracetamol', 'Analgesic', 'Tablet', '500mg', 1000, 0.50,
'2025-12-31'),**

**('Amoxicillin', 'Antibiotic', 'Capsule', '500mg', 500, 2.00, '2024-
10-15'),**

**('lisinopril', 'Inhaler', 'Inhaler', '100mcg', 100, 150.00, '2024-
12-01'),**

**('Fluoxetine', 'Antacid', 'Capsule', '20mg', 300, 1.50, '2025-06-
30');**

-- Sample Queries

-- Query 1: Get all students in Computer Science department

SELECT StudentID, FullName, Phone, YearOfStudy

FROM STUDENT s

JOIN DEPARTMENT d ON s.DepartmentID = d.DepartmentID

WHERE d.DepartmentName = 'Computer Science';

-- Query 2: Count visits by department

SELECT d.DepartmentName, COUNT(v.VisitID) AS VisitCount

FROM VISIT v

JOIN STUDENT s ON v.StudentID = s.StudentID

JOIN DEPARTMENT d ON s.DepartmentID = d.DepartmentID

GROUP BY d.DepartmentName

ORDER BY VisitCount DESC;

-- Query 3: Find medications expiring soon

SELECT Medication Name, Quantity In Stock, Expiry Date,

DATE (ExpiryDate, CURRENT_DATE) AS DaysToExpiry

FROM MEDICATION

WHERE ExpiryDate BETWEEN CURRENT_DATE AND

DATE_ADD(CURRENT_DATE, INTERVAL 90 DAY)

ORDER BY ExpiryDate;

-- Query 4: Get student medical history

```
SELECT s.FullName, v.VisitDate, v.Diagnosis,  
       GROUP_CONCAT(m.MedicationName SEPARATOR ', ') AS  
PrescribedMedications  
FROM VISIT v  
JOIN STUDENT s ON v.StudentID = s.StudentID  
LEFT JOIN PRESCRIPTION p ON v.VisitID = p.VisitID  
LEFT JOIN MEDICATION m ON p.MedicationID =  
m.MedicationID  
WHERE s.StudentID = 301  
GROUP BY v.VisitID  
ORDER BY v.VisitDate DESC;
```

-- Query 5: Check available doctors in morning shift

```
SELECT DoctorID, Name, Specialization, Phone  
FROM DOCTOR  
WHERE WorkingShift = 'Morning' AND Status = 'Active';
```

-- Query 6: Update student phone number

```
UPDATE STUDENT
```

SET Phone = '+251977889900'

WHERE StudentID = 301;

-- Query 7: Delete cancelled visits older than 6 months

DELETE FROM VISIT

WHERE Status = 'Cancelled'

AND VisitDate < DATE_SUB(CURRENT_DATE, INTERVAL 6 MONTH);

-- Query 8: Get prescription details for a visit

**SELECT v.VisitDate, m.MedicationName, p.Dosage,
p.Frequency, p.Duration**

FROM PRESCRIPTION p

JOIN VISIT v ON p.VisitID = v.VisitID

JOIN MEDICATION m ON p.MedicationID = m.MedicationID

WHERE v.VisitID = 1;

-- Query 9: Find pending lab tests

SELECT lt.TestName, s.FullName, lt.RequestedDate, lt.Status


```
FROM LAB_TEST It
JOIN VISIT v ON It.VisitID = v.VisitID
JOIN STUDENT s ON v.StudentID = s.StudentID
WHERE It.Status = 'Pending'
ORDER BY It.RequestedDate;
```

-- Query 10: Get low stock medications

```
SELECT MedicationName, QuantityInStock, ReorderLevel
FROM MEDICATION
WHERE QuantityInStock <= ReorderLevel;
```

-- Constraints Summary

/*

FOREIGN KEYS:

- STUDENT.DepartmentID → DEPARTMENT.DepartmentID
- VISIT.StudentID → STUDENT.StudentID
- VISIT.DoctorID → DOCTOR.DoctorID
- PRESCRIPTION.VisitID → VISIT.VisitID
- PRESCRIPTION.MedicationID → MEDICATION.MedicationID

- LAB_TEST.VisitID → VISIT.VisitID

CHECK Constraints:

- Gender validation (M/F/O)
- Year of study range (1-6)
- Phone number format
- Various status checks
- Quantity validations
- Date validations

UNIQUE Constraints:

- Student Phone
- Doctor LicenseNumber and Phone
- Department Name and Code
- Unique visit slots for doctors
- Unique prescription items per visit

NOT NULL Constraints:

- All primary keys

- Essential attributes like names, dates,

Insertion

```
INSERT INTO Departments (DeptName) VALUES ('Civil',  
( 'Comprehensive'), (Frrshaman'), ('Freshman');
```

```
INSERT INTO Students VALUES ('WCU/101/14', 'Matayisa', 'M',  
1, 'B-04');
```

```
INSERT INTO Doctors (DoctorName, Specialization) VALUES  
( 'Dr.Bethelehem', 'General Practitioner');
```

```
INSERT INTO Medicine (DrugName, Quantity, ExpiryDate)  
VALUES ('Amoxicillin', 100, '2026-12-30');
```

SQL QUERIES FOR WACHEMO UNIVERSITY CLINIC DATABASE

**Query 1: Get all Computer Science students with
their details**

```
SELECT
```

```
StudentID,
```

```
    FullName,  
    Gender,  
FROM STUDENT  
JOIN DEPARTMENT d ON s.DepartmentID =  
d.DepartmentID  
WHERE d.DepartmentName = 'Computer Science'  
ORDER BY FullName;
```

-- Query 2: Get all completed visits in January 2024
with doctor and student info

```
SELECT  
    v.VisitID,  
    s.FullName AS StudentName,  
    d.Name AS DoctorName,  
    v.VisitDate,  
    v.VisitTime,
```

v.VisitType,

v.Diagnosis,

v.Symptoms

FROM VISIT v

JOIN STUDENT s ON v.StudentID = s.StudentID

JOIN DOCTOR d ON v.DoctorID = d.DoctorID

WHERE v.Status = 'Completed'

**AND v.VisitDate BETWEEN '2024-01-01' AND
'2024-01-31'**

ORDER BY v.VisitDate DESC, v.VisitTime DESC;

**-- Query 3: Get all medications that are low in
stock (below reorder level)**

SELECT

MedicationID,

MedicationName,

GenericName,

Category,

QuantityInStock,

ReorderLevel,

UnitPrice,

ExpiryDate,

CASE

**WHEN QuantityInStock = 0 THEN 'OUT OF
STOCK'**

**WHEN QuantityInStock <= ReorderLevel THEN
'LOW STOCK'**

ELSE 'IN STOCK'

END AS StockStatus

FROM MEDICATION

WHERE QuantityInStock <= ReorderLevel

ORDER BY QuantityInStock ASC;

-- Query 4: Get all prescriptions for a specific student (StudentID = 301)

SELECT

p.PrescriptionID,

v.VisitDate,

m.MedicationName,

p.Dosage,

p.Frequency,

p.Duration,

p.QuantityPrescribed,

p.Instructions,

d.Name AS PrescribingDoctor

FROM PRESCRIPTION p

JOIN VISIT v ON p.VisitID = v.VisitID

**JOIN MEDICATION m ON p.MedicationID =
m.MedicationID**

JOIN DOCTOR d ON v.DoctorID = d.DoctorID

WHERE v.StudentID = 301

ORDER BY v.VisitDate DESC;

**-- Query 5: Count visits by department for the
current month**

SELECT

d.DepartmentName,

COUNT(v.VisitID) AS TotalVisits,

**COUNT(DISTINCT v.StudentID) AS
UniqueStudents,**

**AVG(CASE WHEN v.Temperature > 37.5 THEN 1
ELSE 0 END) * 100 AS FeverPercentage**

FROM VISIT v

JOIN STUDENT s ON v.StudentID = s.StudentID


```
JOIN DEPARTMENT d ON s.DepartmentID =  
d.DepartmentID  
  
WHERE MONTH(v.VisitDate) =  
MONTH(CURRENT_DATE)  
  
    AND YEAR(v.VisitDate) = YEAR(CURRENT_DATE)  
  
GROUP BY d.DepartmentName  
  
ORDER BY TotalVisits DESC;
```

-- Query 6: Get all pending lab tests with patient
and doctor information

```
SELECT  
  
    lt.TestID,  
  
    lt.TestName,  
  
    lt.TestType,  
  
    lt.RequestedDate,  
  
    s.FullName AS StudentName,
```

```
d.Name AS RequestingDoctor,  
It.Status,  
DATEDIFF(CURRENT_DATE, It.RequestedDate)  
AS DaysPending  
FROM LAB_TEST It  
JOIN VISIT v ON It.VisitID = v.VisitID  
JOIN STUDENT s ON v.StudentID = s.StudentID  
JOIN DOCTOR d ON v.DoctorID = d.DoctorID  
WHERE It.Status IN ('Pending', 'Sample Collected',  
'In Progress')  
ORDER BY It.RequestedDate ASC;
```

-- Query 7: Find all students with penicillin allergy

```
SELECT  
StudentID,  
FullName,
```

```
DepartmentID,  
YearOfStudy,  
Phone,  
Allergies  
FROM STUDENT  
WHERE Allergies LIKE '%Penicillin%'  
OR Allergies LIKE '%Penicillin%'  
ORDER BY FullName;
```

-- Query 8: Get doctor workload statistics
(appointments per doctor)

```
SELECT  
d.DoctorID,  
d.Name,  
d.Specialization,  
d.WorkingShift,
```

```
COUNT(v.VisitID) AS TotalAppointments,  
COUNT(CASE WHEN v.VisitType = 'Emergency'  
THEN 1 END) AS EmergencyVisits,  
COUNT(CASE WHEN v.Status = 'Completed' THEN  
1 END) AS CompletedVisits,  
ROUND(COUNT(v.VisitID) * 100.0 / (SELECT  
COUNT(*) FROM VISIT), 2) AS PercentageOfTotal  
FROM DOCTOR d  
LEFT JOIN VISIT v ON d.DoctorID = v.DoctorID  
AND v.VisitDate BETWEEN '2024-01-01' AND  
'2024-02-29'  
GROUP BY d.DoctorID, d.Name, d.Specialization,  
d.WorkingShift  
ORDER BY TotalAppointments DESC;
```

-- Query 9: Get medication usage statistics

```
SELECT
```

```
m.MedicationName,  
m.Category,  
m.QuantityInStock,  
SUM(p.QuantityPrescribed) AS TotalPrescribed,  
COUNT(p.PrescriptionID) AS TimesPrescribed,  
m.UnitPrice,  
m.UnitPrice * SUM(p.QuantityPrescribed) AS  
TotalValuePrescribed  
FROM MEDICATION m  
LEFT JOIN PRESCRIPTION p ON m.MedicationID =  
p.MedicationID  
GROUP BY m.MedicationID, m.MedicationName,  
m.Category, m.QuantityInStock, m.UnitPrice  
ORDER BY TotalPrescribed DESC;
```

**-- Query 10: Find students with multiple visits
(frequent visitors)**

SELECT

s.StudentID,

s.FullName,

d.DepartmentName,

s.YearOfStudy,

COUNT(v.VisitID) AS VisitCount,

MIN(v.VisitDate) AS FirstVisit,

MAX(v.VisitDate) AS LastVisit,

**GROUP_CONCAT(DISTINCT v.Diagnosis
SEPARATOR ', ') AS CommonDiagnoses**

FROM STUDENT s

JOIN VISIT v ON s.StudentID = v.StudentID

**JOIN DEPARTMENT d ON s.DepartmentID =
d.DepartmentID**

**GROUP BY s.StudentID, s.FullName,
d.DepartmentName, s.YearOfStudy**

HAVING COUNT(v.VisitID) > 1

ORDER BY VisitCount DESC;

-- SECTION 2: UPDATE QUERIES (Data Modification)

-- Update 1: Update a student's phone number

UPDATE STUDENT

SET Phone = '+251944112233',

Email = 'new.email@wachemo.edu.et'

WHERE StudentID = 301;

-- Update 2: Mark a prescription as dispensed

UPDATE PRESCRIPTION

SET IsDispensed = TRUE

WHERE PrescriptionID = 5;

-- Update 3: Update lab test result and status

```
UPDATE LAB_TEST  
SET Result = 'Normal',  
    ResultValue = 4.2,  
    Status = 'Completed',  
    ReportDate = CURRENT_DATE,  
    LabTechnician = 'Tech. Martha H.'  
WHERE TestID = 3;
```

-- Update 4: Update medication stock after dispensing

```
UPDATE MEDICATION  
SET QuantityInStock = QuantityInStock - 10  
WHERE MedicationID = 1;
```

-- Update 5: Change doctor's working shift

```
UPDATE DOCTOR  
SET WorkingShift = 'Afternoon',
```


Status = 'Active'

WHERE DoctorID = 201;

-- Update 6: Update visit diagnosis and symptoms

UPDATE VISIT

SET Diagnosis = 'Viral Infection',

Symptoms = 'Fever, cough, fatigue',

Temperature = 38.2

WHERE VisitID = 10;

-- Update 7: Update student's allergy information

UPDATE STUDENT

WHERE StudentID = 302;

-- Update 8: Update medication price (increase by 10%)

UPDATE MEDICATION

SET UnitPrice = UnitPrice * 1.10

WHERE Category = 'Antibiotic';

-- SECTION 3: DELETE QUERIES (Data Removal)

-- Delete 1: Delete a specific prescription (if made in error)

DELETE FROM PRESCRIPTION

WHERE PrescriptionID = 15;

-- Delete 2: Delete cancelled visits older than 1 year

DELETE FROM VISIT

WHERE Status = 'Cancelled'

**AND VisitDate < DATE_SUB(CURRENT_DATE,
INTERVAL 1 YEAR);**

-- Delete 3: Delete expired medications

DELETE FROM MEDICATION

WHERE ExpiryDate < CURRENT_DATE;

-- Delete 4: Delete test records with invalid visit reference

DELETE FROM LAB_TEST

WHERE VisitID NOT IN (SELECT VisitID FROM VISIT);

-- Delete 5: Delete student records for graduated students (with no recent visits)

DELETE FROM STUDENT

WHERE StudentID NOT IN (

SELECT DISTINCT StudentID

FROM VISIT

**WHERE VisitDate > DATE_SUB(CURRENT_DATE,
INTERVAL 2 YEAR)**

)

AND YearOfStudy > 4; -- Assuming they might have graduated

-- SECTION 4: COMPLEX QUERIES WITH JOINS AND SUB QUERIES

-- Query 11: Find students who visited but were not prescribed any medication

SELECT

s.StudentID,

s.FullName,

v.VisitDate,

v.Diagnosis,

d.Name AS Doctor

FROM VISIT v

JOIN STUDENT s ON v.StudentID = s.StudentID

JOIN DOCTOR d ON v.DoctorID = d.DoctorID

WHERE v.VisitID NOT IN (

SELECT DISTINCT VisitID

FROM PRESCRIPTION

)

AND v.Status = 'Completed'

ORDER BY v.VisitDate DESC;

**-- Query 12: Find most commonly prescribed
medication combinations**

SELECT

m1.MedicationName AS Medication1,

m2.MedicationName AS Medication2,

COUNT(*) AS Frequency

FROM PRESCRIPTION p1

JOIN PRESCRIPTION p2 ON p1.VisitID = p2.VisitID

AND p1.MedicationID < p2.MedicationID

**JOIN MEDICATION m1 ON p1.MedicationID =
m1.MedicationID**

**JOIN MEDICATION m2 ON p2.MedicationID =
m2.MedicationID**

**GROUP BY m1.MedicationName,
m2.MedicationName**

HAVING COUNT(*) > 1

ORDER BY Frequency DESC;

**-- Query 13: Find students with chronic conditions
(multiple visits for same diagnosis)**

SELECT

s.StudentID,

s.FullName,

v.Diagnosis,

COUNT(v.VisitID) AS VisitCount,

MIN(v.VisitDate) AS FirstVisit,

MAX(v.VisitDate) AS LastVisit

**FROM VISIT v JOIN STUDENT s ON v.StudentID =
s.StudentID**

WHERE v.Diagnosis IS NOT NULL

AND v.Diagnosis != ''

GROUP BY s.StudentID, s.FullName, v.Diagnosis

HAVING COUNT(v.VisitID) > 2

ORDER BY VisitCount DESC;

-- Query 14: Get department-wise health statistics

SELECT

dep.DepartmentName,

COUNT(DISTINCT s.StudentID) AS TotalStudents,

COUNT(v.VisitID) AS TotalVisits,

**ROUND(COUNT(v.VisitID) * 1.0 /
COUNT(DISTINCT s.StudentID), 2) AS
VisitsPerStudent,**

**GROUP_CONCAT(DISTINCT v.Diagnosis ORDER
BY v.Diagnosis SEPARATOR '; ') AS
CommonDiagnoses**

```
FROM DEPARTMENT dep
LEFT JOIN STUDENT s ON dep.DepartmentID =
s.DepartmentID
LEFT JOIN VISIT v ON s.StudentID = v.StudentID
    AND v.VisitDate >= '2024-01-01'
GROUP BY dep.DepartmentID,
dep.DepartmentName
ORDER BY TotalVisits DESC;

-- Query 15: Find medications about to expire in
the next 30 days

SELECT
    MedicationName,
    QuantityInStock,
    ExpiryDate,
    DATEDIFF(ExpiryDate, CURRENT_DATE) AS
DaysToExpiry,
    CASE
```


**WHEN DATEDIFF(ExpiryDate, CURRENT_DATE)
<= 0 THEN 'EXPIRED'**

**WHEN DATEDIFF(ExpiryDate, CURRENT_DATE)
<= 30 THEN 'EXPIRING SOON'**

ELSE 'VALID'

END AS ExpiryStatus

FROM MEDICATION

**WHERE ExpiryDate BETWEEN CURRENT_DATE AND
DATE_ADD(CURRENT_DATE, INTERVAL 30 DAY)**

ORDER BY ExpiryDate;

**-- SECTION 5: TRANSACTION EXAMPLES WITH
ROLLBACK**

-- Example 1: Safe update with transaction

START TRANSACTION;

-- Backup current data

**CREATE TEMPORARY TABLE IF NOT EXISTS
student_backup AS**

SELECT * FROM STUDENT WHERE StudentID = 301;

-- Update student information

UPDATE STUDENT

SET Phone = '+251977889900',

Email = 'updated@wachemo.edu.et'

WHERE StudentID = 301;

-- Verify the update

SELECT * FROM STUDENT WHERE StudentID = 301;

-- If something went wrong, we can rollback

-- ROLLBACK;

-- If everything is okay, commit

COMMIT;

**-- Example 2: Delete with referential integrity
check**

START TRANSACTION;

-- Check if student has any records in other tables

SELECT

**(SELECT COUNT(*) FROM VISIT WHERE
StudentID = 315) AS VisitCount,**

**(SELECT COUNT(*) FROM PRESCRIPTION p JOIN
VISIT v ON p.VisitID = v.VisitID WHERE v.StudentID
= 315) AS PrescriptionCount;**

-- If counts are 0, safe to delete

DELETE FROM STUDENT WHERE StudentID = 315;

COMMIT;

**-- SECTION 6: AGGREGATION AND REPORTING
QUERIES**

-- Query 16: Monthly visit statistics report

SELECT

DATE_FORMAT(VisitDate, '%Y-%m') AS Month,

```
COUNT(*) AS TotalVisits,  
COUNT(DISTINCT StudentID) AS UniquePatients,  
COUNT(CASE WHEN VisitType = 'Emergency'  
THEN 1 END) AS EmergencyVisits,  
COUNT(CASE WHEN VisitType = 'Follow-up'  
THEN 1 END) AS FollowUpVisits,  
ROUND(AVG(Temperature), 2) AS  
AvgTemperature  
FROM VISIT  
WHERE VisitDate BETWEEN '2024-01-01' AND  
'2024-02-29'  
GROUP BY DATE_FORMAT(VisitDate, '%Y-%m')  
ORDER BY Month;
```

-- Query 17: Top 5 most common diagnoses

```
SELECT  
Diagnosis,
```

```
COUNT(*) AS Frequency,  
COUNT(DISTINCT StudentID) AS UniquePatients,  
ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*)  
FROM VISIT WHERE Diagnosis IS NOT NULL), 2) AS  
Percentage  
FROM VISIT  
WHERE Diagnosis IS NOT NULL  
AND Diagnosis != ''  
GROUP BY Diagnosis  
ORDER BY Frequency DESC  
LIMIT 5;
```

-- Query 18: Doctor performance report

```
SELECT  
d.DoctorID,  
d.Name,
```

```
d.Specialization,  
COUNT(v.VisitID) AS TotalVisits,  
AVG(TIMESTAMPDIFF(MINUTE,  
CONCAT(v.VisitDate, ' ', v.VisitTime),  
CURRENT_TIMESTAMP)) AS AvgConsultationTime,  
COUNT(DISTINCT v.Diagnosis) AS  
UniqueDiagnoses,  
COUNT(p.PrescriptionID) AS  
PrescriptionsWritten  
FROM DOCTOR d  
LEFT JOIN VISIT v ON d.DoctorID = v.DoctorID  
LEFT JOIN PRESCRIPTION p ON v.VisitID = p.VisitID  
GROUP BY d.DoctorID, d.Name, d.Specialization  
ORDER BY TotalVisits DESC;
```

```
-- Query 19: Medication inventory value report  
SELECT
```

```
Category,  
COUNT(*) AS MedicationCount,  
SUM(QuantityInStock) AS TotalUnits,  
ROUND(AVG(UnitPrice), 2) AS AvgUnitPrice,  
SUM(QuantityInStock * UnitPrice) AS TotalValue,  
MIN(ExpiryDate) AS EarliestExpiry,  
SUM(CASE WHEN QuantityInStock <=  
ReorderLevel THEN 1 ELSE 0 END) AS  
LowStockItems  
FROM MEDICATION  
GROUP BY Category  
ORDER BY TotalValue DESC;
```

-- Query 20: Student health profile summary

```
SELECT
```

```
    s.StudentID,
```

**s.FullName,
d.DepartmentName,
s.YearOfStudy,
s.BloodGroup,
s.Allergies,
COUNT(v.VisitID) AS TotalVisits,
MIN(v.VisitDate) AS FirstVisit,
MAX(v.VisitDate) AS LastVisit,
GROUP_CONCAT(DISTINCT v.Diagnosis ORDER
BY v.VisitDate DESC SEPARATOR ', ') AS
MedicalHistory
FROM STUDENT s
LEFT JOIN VISIT v ON s.StudentID = v.StudentID
JOIN DEPARTMENT d ON s.DepartmentID =
d.DepartmentID**

**GROUP BY s.StudentID, s.FullName,
d.DepartmentName, s.YearOfStudy, s.BloodGroup,
s.Allergies**

ORDER BY TotalVisits DESC;

Constraints

Constraints are rules used to ensure the accuracy and reliability of the data within the Wachemo University Clinic database. They prevent "garbage data" from being entered into student records.

- * PRIMARY KEY (PK): Uniquely identifies each record.**
 - * Example: StudentID in the Students table ensures that no two students have the same ID number.**
- * FOREIGN KEY (FK): Maintains "Referential Integrity" between tables.**
 - * Example: The DeptID in the Students table must exist in the Departments table. This prevents a student**

from being assigned to a department that does not exist at Wachemo University.

- * NOT NULL:** Ensures that critical fields are never left empty.

- * Example:** The Diagnosis field in MedicalRecords must be filled so that a student's medical history is never missing.

- * UNIQUE:** Ensures all values in a column are different.

- * Example:** A student's Phone_Number or Email should be unique to avoid contacting the wrong person regarding their health.

- * CHECK:** Ensures data falls within a specific range or category.

- * Example:** CHECK (Gender IN ('M', 'F')) ensures that only 'M' or 'F' can be entered into the system.

Results and Discussion

This section explains how the database actually performs when used by clinic staff to manage student

services. We use SQL queries to generate results and discuss their importance.

Query 1: List all students in the 'Information Technology' department

*** SQL Code: SELECT FullName, StudentID FROM Students WHERE DeptID = (SELECT DeptID FROM Departments WHERE DeptName = 'Information Technology');**

*** Result Discussion: This result allows the clinic to identify how many IT students are using the facility. This is helpful for health statistics or for targeting health awareness campaigns to specific student groups.**

Query 2: Find total number of students treated today

*** SQL Code: SELECT COUNT(RecordID) FROM MedicalRecords WHERE VisitDate = CURRENT_DATE;**

*** Result Discussion: This helps university management understand the daily "load" or traffic at the clinic. If the number is very high (e.g., over 100), the university might decide to assign more doctors to that shift.**

Query 3: Show out-of-stock medicines (Quantity < 5)

*** SQL Code: SELECT DrugName, Quantity FROM Medicine WHERE Quantity < 5;**

*** Result Discussion: This is a vital service for the Pharmacy. It acts as an "alert system" so that the pharmacist can order new supplies before the clinic completely runs out of essential medication like Paracetamol or Antibiotics.**

Final Submission Checklist

Based on your provided guidelines, ensure you submit:

- * Project Report: Save as a PDF or DOCX file.**
- * SQL File: Save all your code as a .sql file.**
- * ER Diagram: Include a clear, readable image of your diagram.**

Would you like me to provide the "Conclusion" (Section 15) to complete your entire report?